Name: Ryaan Devnani	Class: T1	Roll No: 2401023
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

Aim: To perform version control operations using Git.

LO 02: Introduction to Git, GitHub, Basic operations: clone, commit, push, pull, Branching strategies and merging, Git workflows: Feature branching, Git Flow, Handling merge conflicts, GitHub Actions basics

Git Commands — Version Control Experiment

Setup & Configuration

1. git --version — Checks which version of Git is installed on the system.

2. git config --global user.name “Ryaan Devnani" — Sets the username attached to your commits.

3. git config --global user.email “ryaandevnani@gmail.com” — Sets the email attached to your commits.

Creating the Repository
4. mkdir git_experiment — Creates a new folder for the project.
5. cd git_experiment — Moves into that folder.
6. git init — Initializes a new Git repository in the current folder.
7. git status — Shows the current state of the working directory (tracked/untracked/staged files).

Creating & Committing Files
8. echo 'print("Hello World - Version 1")' > app.py — Creates a Python file with starter code.
9. git add app.py — Stages a specific file, marking it ready to be committed.
10. git commit -m "message" — Saves a permanent snapshot of staged changes with a message.
11. git log --oneline — Displays commit history in a short, one-line-per-commit format.

Editing & Comparing Versions
12. echo 'print("Hello World - Version 2")' > app.py — Edits the file to create a new version.
13. git diff — Shows line-by-line differences between the working file and the last commit.
14. git add . — Stages all changed/new files in the folder at once.
15. git commit -m "message" — Commits the newly staged changes.

Branching
16. git branch — Lists all branches and shows which one is currently active.
17. git branch feature-branch — Creates a new branch without switching to it.
18. git checkout feature-branch — Switches the working directory to the specified branch.
19. echo 'print(...)' > app.py — Edits the file while on the new branch.
20. git commit -am "message" — Stages tracked files and commits in a single step.
21. git checkout master — Switches back to the main branch.
22. git merge feature-branch — Combines changes from feature-branch into the current branch.
23. git checkout -b test-branch — Creates and switches to a new branch in one command.

Undoing Changes / Rollback
24. git restore app.py — Discards uncommitted changes in a file, reverting it to the last commit.
25. git add app.py — Stages a file (used before demonstrating unstage below).
26. git restore --staged app.py — Unstages a file without deleting the actual edit.
27. git reset --soft HEAD~1 — Undoes the last commit but keeps changes staged.
28. git reset --hard HEAD~1 — Completely undoes the last commit and discards the changes (destructive).
29. git revert <commit-id> — Creates a new commit that undoes an older commit's changes, without deleting history (the safe rollback method).

Viewing Final State
30. git log --oneline — Final check showing the complete commit history after all operations.

git —version

```
(base) ryaandevnani@Ryaans-MacBook-Air ~ % git --version
git version 2.47.0
(base) ryaandevnani@Ryaans-MacBook-Air ~ %
```

gitconfig --global user.name "Ryaan Devnani”

```
(base) ryaandevnani@Ryaans-MacBook-Air ~ % git config --global user.name "Ryaan Devnani"
```

git config --global user.email "ryaandevnani@gmail.com"

```
(base) ryaandevnani@Ryaans-MacBook-Air ~ % git config --global user.email "ryaandevnani@gmail.com"
```

mkdir git_experiment cd git_experiment

```
(base) ryaandevnani@Ryaans-MacBook-Air ~ % mkdir git_experiment
```

cd git_experiment

```
(base) ryaandevnani@Ryaans-MacBook-Air ~ % cd git_experiment
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment %
```

git init

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /Users/ryaandevnani/git_experiment/.git/
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment %
```

git status

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

echo 'print("Hello World - Version 1")' > app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % echo 'print("Hello World - Version 1")' > app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % cat app.py
print("Hello World - Version 1")
```

git add app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git add app.py
```

git commit -m "Version 1: initial hello world"

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git commit -m "Version 1: initial hello world"

[master (root-commit) db9b512] Version 1: initial hello world
 1 file changed, 1 insertion(+)
 create mode 100644 app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment %
```

git log --oneline

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git log --oneline
db9b512 (HEAD -> master) Version 1: initial hello world
```

echo 'print("Hello World - Version 2")' > app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % echo 'print("Hello World - Version 2")' > app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % cat app.py
print("Hello World - Version 2")
```

git diff

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git diff
diff --git a/app.py b/app.py
index 92e9091..e1b1d0e 100644
--- a/app.py
+++ b/app.py
@@ -1 +1 @@
-print("Hello World - Version 1")
+print("Hello World - Version 2")
```

git commit -m "Version 2: updated message"

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git commit -m "Version 2: updated message"
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

git log --oneline

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git log --oneline

454a9df (HEAD -> master) Version 2: updated message
db9b512 Version 1: initial hello world
```

git branch feature-branch

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git branch feature-branch
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git branch
  feature-branch
* master
```

git checkout feature-branch

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git checkout feature-branch
Switched to branch 'feature-branch'
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git branch
* feature-branch
  master
```

echo 'print("Hello World - Version 3 on feature branch")' > app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % echo 'print("Hello World - Version 3 on feature branch")' > app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % cat app.py
print("Hello World - Version 3 on feature branch")
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git status
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

git commit -am "Version 3: edited on feature branch”

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git commit -am "Version 3: edited on feature branch"
[feature-branch 329dea7] Version 3: edited on feature branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

git checkout master

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git checkout master
Switched to branch 'master'
```

git merge feature-branch

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git merge feature-branch
Updating 454a9df..329dea7
Fast-forward
 app.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

git checkout -b test-branch

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git checkout -b test-branch
Switched to a new branch 'test-branch'
```

git checkout master

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git checkout master
Switched to branch 'master'
```

echo temp change > app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % echo temp change > app.py
```

echo temp change > app.py
git restore app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git restore app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % cat app.py
print("Hello World - Version 3 on feature branch")
```

echo another temp change > app.py

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % echo another temp change > app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git add app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git restore --staged app.py
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

git add app.py
git commit -m "temp commit for reset demo"

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git add app.py

(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git commit -m "temp commit for reset demo"

[master 7efd2bc] temp commit for reset demo
 1 file changed, 1 insertion(+), 1 deletion(-)
```

git reset --soft HEAD~1
git status

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git reset --soft HEAD~1

(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

git commit -m "temp commit for hard reset demo"

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git commit -m "temp commit for hard reset demo"

[master 3e917e5] temp commit for hard reset demo
 1 file changed, 1 insertion(+), 1 deletion(-)
```

git reset --hard HEAD~1

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git reset --hard HEAD~1

HEAD is now at 329dea7 Version 3: edited on feature branch
```

git status

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git status

On branch master
nothing to commit, working tree clean
```

git log —oneline

```
(base) ryaandevnani@Ryaans-MacBook-Air git_experiment % git log --oneline
4c8f272 (HEAD -> master) Revert "Version 3: edited on feature branch"
329dea7 (test-branch, feature-branch) Version 3: edited on feature branch
454a9df Version 2: updated message
db9b512 Version 1: initial hello world
```

Conclusion: This experiment provided practical experience with Git version control, covering the complete workflow from initializing a repository and committing changes to branching, merging, and rolling back versions using restore, reset, and revert. By creating multiple versions of a Python file and experimenting with commands like git log, git diff, and resolving a real merge conflict during a revert, I gained a clear understanding of how Git tracks project history and allows developers to safely undo or recover previous states of their code, which is essential for reliable collaborative software development.